

LECTURE · 量化金融 365

# DAY 031 / 365

P2 · PYTHON 量化工具栈

DEEP DIVE · 8 页深度版

## 环境搭建:Anaconda 工作流

// Setup

KEY CONCEPTS · 三大要点

- ▶ Anaconda
- ▶ 虚拟环境
- ▶ VS Code/Jupyter



01 · WHY THIS LESSON · 这节课为什么重要

## 本节定调

// 看完这一段你会知道:这节课跟你接下来 3 个月的学习节奏关系有多大

今天是 P2 工程化阶段的第一节。前 29 天我们写过的所有 Python 代码,环境基本是「装个 Python 直接开搞」。这种做法在写一两个小脚本时没问题,做正经量化项目立刻翻车。今天讲清楚四件事 — 第一,为什么必须用虚拟环境而不是直接装包到系统 Python,讲一个让所有工程师都心有余悸的真实故事;第二,Anaconda 跟 Miniconda、pip 跟 poetry 这几个工具到底什么差别,量化场景里怎么选;第三,量化必装的 15 个包清单 + 国内镜像源配置,装好之后你的环境跟头部私募研究员的工作环境基本一致;第四,Jupyter Lab 跟 VS Code 各自适合什么场景、什么时候用哪个。最后给你一个标准的量化项目目录结构 + .gitignore 模板,直接抄走就行。学完这一节你的工作环境就升级到了「专业研究员」水平,后面 P2 P3 P4 所有项目都站在这个基础上。

学习目标 · LEARNING OBJECTIVES

// 听课前默念一遍,听完之后回头打勾

- 01 理解为什么必须用虚拟环境 — 通过一个真实的「包冲突」事故,看清楚不隔离的代价
- 02 区分 Anaconda / Miniconda / pip / poetry 四个工具的定位,知道量化场景里怎么选
- 03 搭建一个标准的量化研究环境 — 一行命令装好 15 个核心包,配国内镜像源加速 10 倍
- 04 选对你的主战场 IDE — Jupyter Lab(研究 / 探索 / 出报告)+ VS Code(写库 / 调试 / 部署)双轨制
- 05 学会一套标准的量化项目目录结构 + .gitignore 模板,直接复用到所有后续项目

02 · CONTEXT · 历史与背景

# 一个未隔离环境引发的连锁事故 — 量化新手的噩梦

// 不读历史的策略走不远

讲一个真实案例。某私募研究员小张,2022 年刚入职,直接用电脑自带的 Python 装包。一开始没问题,他装了 pandas、numpy、matplotlib,做了几个回测项目,跑得挺顺。

3 个月后,公司接了个新项目要做深度学习因子,他装了 tensorflow 2.10。装完那一刻没事,第2天打开之前的回测项目,所有代码都跑不起来了。报错信息一行接一行,提示 numpy 版本不兼容。原因是 tensorflow 2.10 要求 numpy 1.23,而他之前的回测项目用的是 numpy 1.21,旧 pandas 也只支持 numpy 1.21。tensorflow 装的时候自动把 numpy 升级了,把整个旧项目环境砸了。

他想退回去,pip uninstall numpy 然后 pip install numpy==1.21,结果 tensorflow 又跑不起来了。这次他想分别处理,把回测项目跟深度学习项目放不同目录,但代码运行的还是同一个 Python,同一套包,根本不存在「分别」。

他卡在这里两天,改了一堆代码强行妥协,最后发现总有一些库怎么都协调不了。第3天他去找 CTO,CTO 说一句话 — 你为什么不用 conda 虚拟环境?他当场愣住,因为他从来不知道虚拟环境是什么,以为「装 Python = 装一个全局环境」就是全部。

CTO 花了 20 分钟教他 conda create / conda activate,他当晚把所有项目分别建独立环境,问题立刻解决。回头看,这两天他白损失了 16 小时工作,差点搞砸两个项目交付。

这个故事的核心教训不是 conda 多牛 — 是「不隔离 = 不可持续」。任何写过 1 年以上 Python 的人都遇到过类似事故。虚拟环境不是高级功能,是基础设施。今天这一节我们就讲清楚怎么搭、为什么搭、怎么避免小张的痛苦。这一节学完你就不会再踩这个坑,后面 P2 P3 都顺利利。

03 · CORE CONCEPTS · 核心概念详解

# 把这几个概念彻底搞懂

// 听完视频后,确保你能给一个完全不懂的朋友讲清楚每一条

## CONCEPT\_01

### 为什么必须用虚拟环境 — 隔离的工程价值

虚拟环境的核心价值,可以用一个简单的类比说清楚 — 工作环境就像你的房间,装的包就像你买的家具。如果所有项目共用一个房间(系统 Python),家具会越堆越多,有的家具尺寸冲突放不下,有的家具材质冲突放在一起会出问题。你最后只能在一个房间里凑合,所有项目都将就。

虚拟环境就是给每个项目一个独立的房间。回测项目一个房间装回测要用的家具(pandas 1.5、numpy 1.21、matplotlib 3.5),机器学习项目另一个房间装机器学习要用的家具(pandas 2.0、numpy 1.23、tensorflow 2.10),互不打扰。

这个隔离带来三个具体好处。第一,版本冲突彻底解决。每个项目锁定它自己的包版本,旧项目永远跑得动,新项目用最新版本。第二,可复现性。把你的环境配置(requirements.txt 或 environment.yml)分享给别人,别人 5 分钟内能在自己电脑上复制一个一模一样的环境,跑出同样的结果。第三,清理方便。某个项目不要了直接删整个环境,系统没有任何残留。系统 Python 装多了根本删不干净。

业内规则非常简单 — 任何超过 100 行代码的项目都必须用虚拟环境。低于 100 行的小脚本也建议用,只是不强求。专业团队里,「不用虚拟环境」基本等于「不专业」,跟数据科学家说「我直接 pip install 到系统 Python」对方会立刻怀疑你的工程能力。

虚拟环境是工程化的第一道关。过了这一关你才算正式入门,接下来的 Git、测试、CI 都建立在这一关之上。

## CONCEPT\_02

### Anaconda / Miniconda / pip / poetry — 怎么选

Python 包管理工具有很多种,新手最容易迷茫的就是「这么多工具我该用哪个」。下面我把量化场景里最常用的四个工具拆开讲,你看完就知道怎么选。

Anaconda — 一站式数据科学发行版。安装包 500MB+,装好之后自带 Python + 200 多个常用包 + Jupyter + 一个图形界面工具 Anaconda Navigator。优点是「装好就能用」,新手友好。缺点是体积大、占硬盘、自带很多你用不上的包。建议没碰过的新手第一次装 Anaconda,熟悉了之后切换到 Miniconda。

03 · CORE CONCEPTS · 续 (2/3)

CONCEPT\_03

## 必装包清单 + 国内镜像源

下面是量化研究员的「最低限度装包清单」,15 个包,一行命令装完。

数据处理(5 个) — numpy 数值计算基础、pandas 数据表操作、scipy 科学计算、statsmodels 统计建模、scikit-learn 机器学习。这 5 个是任何数据相关的基础。

可视化(3 个) — matplotlib 静态绘图基础、seaborn 高级统计图、plotly 交互式图表。matplotlib 必装,其他两个按需。

数据接口(3 个) — yfinance 海外免费、akshare 国内免费零翻墙、tushare 国内专业(可选)。这 3 个是数据来源主力,国内用户 akshare 必装,海外用户 yfinance 必装。

量化专用(4 个) — quantstats 一键策略报告、empyrical 指标计算、ffn 指标可视化一体、pyfolio 风险归因。这 4 个是后面 D29 那种「策略报告六件套」的快捷工具。

安装命令 — 一行搞定。

```
conda create -n quant python=3.11 -y
```

```
conda activate quant
```

```
pip install numpy pandas scipy statsmodels scikit-learn matplotlib seaborn plotly  
yfinance akshare quantstats empyrical ffn pyfolio-reloaded jupyter
```

国内镜像源加速 — pip 默认从 PyPI 海外下载,在国内速度可能慢 10 倍。配置清华镜像源后变得飞快。

```
pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple
```

conda 仓库也要配国内镜像源。在 ~/.condarc 文件里加上清华源(具体配置网上搜「conda 清华源」)。配完之后 conda install 也飞快。

这套环境装好之后,你就有了一台「能干 90% 量化工作」的电脑。占硬盘 5GB 左右,装一次能用很久。

CONCEPT\_04

## Jupyter Lab vs VS Code — 双轨制

IDE 选哪个是量化新手第二个迷茫点。我直接告诉你结论 — 不要选,两个都用。Jupyter Lab 跟 VS Code 各自适合不同场景,组合起来才是完整 workflow。

03 · CORE CONCEPTS · 续 (3/3)

CONCEPT\_05

## 量化项目目录结构 + .gitignore 模板

一个量化项目的标准目录结构,直接抄就行。

```
my_quant_project/
├─ README.md(项目说明)
├─ requirements.txt(依赖清单)
├─ environment.yml(conda 环境配置)
├─ .gitignore(忽略文件配置)
├─ data/(原始数据)
│   └─ raw/(下载下来的原始数据)
│   └─ processed/(清洗后的数据)
├─ notebooks/(研究探索)
│   └─ 01_data_exploration.ipynb
│   └─ 02_strategy_test.ipynb
│   └─ 03_results_analysis.ipynb
├─ src/(核心代码模块)
│   └─ data/(数据获取与清洗)
│   └─ strategy/(策略逻辑)
│   └─ backtest/(回测引擎)
│   └─ utils/(工具函数)
├─ tests/(单元测试)
├─ reports/(策略报告输出)
│   └─ figures/(图表)
│   └─ pdf/(PDF 报告)
└─ results/(回测结果)
```

这个结构覆盖了 80% 的量化项目需求。data 跟 notebooks 是研究阶段主战场,src 是把研究成果固化的代码,reports 是给老板看的输出。

.gitignore 模板 — 防止把不该提交的东西提交到 git,这些必须忽略。

```
# Python
__pycache__/
*.pyc
*.pyo
```

04 · PYTHON LAB · 真实可跑代码

# 量化环境自检 — Python / 包 / 性能 / 中文字体 / 镜像源 全面体检

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

► **实操原则:** 先跑通(哪怕硬编码),再优化(参数化/函数化),最后才追求精度(模型升级/特征工程)。散户量化最大的坑是没跑通就开始优化。

```
# day_031_env_check.py — 量化环境完整体检脚本
# 检查:Python 版本 / conda 信息 / 关键包版本 / 性能基准 / 中文字体 / 镜像源
import sys
import os
import platform
import subprocess
import time
import warnings
warnings.filterwarnings('ignore')

import numpy as np
import matplotlib.pyplot as plt

print('==== 1. 系统与 Python 信息 ====')
print(f'操作系统:{platform.system()} {platform.release()}')
print(f'机器架构:{platform.machine()}')
print(f'Python 版本:{sys.version.split()[0]}')
print(f'Python 路径:{sys.executable}')

# ===== 2. Conda 环境信息 =====
print('\n==== 2. Conda 环境信息 ====')
try:
    result = subprocess.run(['conda', 'info', '--envs'], capture_output=True,
                             text=True, timeout=5)
    if result.returncode == 0:
```

04 · PYTHON LAB · 真实可跑代码 · 续 (2/8)

# 量化环境自检 — Python / 包 / 性能 / 中文字体 / 镜像源 全面体检

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
envs_lines = result.stdout.strip().split('\n')
envs = [l for l in envs_lines if l and not l.startswith('#')]
print(f'Conda 检测到 {len(envs)} 个虚拟环境')
for env in envs[:5]:
    print(f' {env}')
else:
    print('Conda 未安装或未在 PATH 中。建议安装 Miniconda。')
except (FileNotFoundError, subprocess.TimeoutExpired):
    print('Conda 未安装或未在 PATH 中。建议安装 Miniconda。')

current_env = os.environ.get('CONDA_DEFAULT_ENV', os.environ.get('VIRTUAL_ENV',
    '系统 Python(未隔离)'))
print(f'当前活跃环境:{current_env}')

# ===== 3. 关键包版本检查 =====
print('\n==== 3. 量化核心包版本 ====')
packages = [
    ('numpy', 'NumPy 数值计算'),
    ('pandas', 'Pandas 数据表'),
    ('scipy', 'SciPy 科学计算'),
    ('matplotlib', 'Matplotlib 绘图'),
    ('sklearn', 'Scikit-learn 机器学习'),
    ('yfinance', 'yfinance 海外数据'),
    ('akshare', 'akshare 国内数据'),
    ('statsmodels', 'StatsModels 统计建模'),
    ('jupyter', 'Jupyter 笔记本'),
]

installed, missing = [], []
for pkg, desc in packages:
    try:
```

04 · PYTHON LAB · 真实可跑代码 · 续 (3/8)

# 量化环境自检 — Python / 包 / 性能 / 中文字体 / 镜像源 全面体检

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
if pkg == 'sklearn':
    import sklearn
    ver = sklearn.__version__
elif pkg == 'jupyter':
    ver = subprocess.run(['jupyter', '--version'], capture_output=True, text=True,
        timeout=3).stdout.split('\n')[0] or '已安装'
else:
    mod = __import__(pkg)
    ver = getattr(mod, '__version__', '未知')
    installed.append((pkg, desc, ver))
print(f' ✓ {pkg:13s} {ver:15s} {desc}')
except (ImportError, FileNotFoundError, subprocess.TimeoutExpired):
    missing.append((pkg, desc))
print(f' ✗ {pkg:13s} {"未安装":15s} {desc}')

print(f'\n安装完成 {len(installed)} 个 / 待安装 {len(missing)} 个')
if missing:
    pkg_names = [p for p, _ in missing]
    print(f'建议安装: pip install {" ".join(pkg_names)}')

# ===== 4. NumPy 性能基准 =====
print('\n==== 4. NumPy 性能基准 ====')
N = 1_000_000

# Python 列表版
t0 = time.perf_counter()
py_list = list(range(N))
py_squares = [x * x for x in py_list]
t_list = time.perf_counter() - t0

# NumPy 版
```

04 · PYTHON LAB · 真实可跑代码 · 续 (4/8)

# 量化环境自检 — Python / 包 / 性能 / 中文字体 / 镜像源 全面体检

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
t0 = time.perf_counter()
np_arr = np.arange(N)
np_squares = np_arr * np_arr
t_np = time.perf_counter() - t0

speedup = t_list / t_np if t_np > 0 else 0
print(f'计算 {N:,} 个数的平方:')
print(f' Python 列表:{t_list*1000:7.1f} ms')
print(f' NumPy 数组:{t_np*1000:7.1f} ms')
print(f' NumPy 加速比:{speedup:.0f} 倍')

# ===== 5. 中文字体检查 =====
print('\n==== 5. Matplotlib 中文字体检查 ====')
from matplotlib.font_manager import fontManager
cn_fonts = []
for f in fontManager.ttflist:
    name = f.name
    if any(k in name for k in ['CJK', 'Hei', 'Song', 'Kai', 'Yi', 'Sim', 'WenQuanYi',
                              'PingFang', 'Noto Sans CJK', 'Noto Serif CJK']):
        if name not in cn_fonts:
            cn_fonts.append(name)
        if cn_fonts:
            print(f'检测到 {len(cn_fonts)} 个中文字体:')
            for f in cn_fonts[:5]:
                print(f' ✓ {f}')
            else:
                print(' ✗ 未检测到中文字体!画图标题会显示方块。')
            print(' 解决:Windows 系统自带 SimHei,Mac 自带 PingFang,Linux 装 fonts-noto-cjk。')

# ===== 6. 镜像源检查 =====
print('\n==== 6. pip 镜像源检查 ====')
```

04 · PYTHON LAB · 真实可跑代码 · 续 (5/8)

# 量化环境自检 — Python / 包 / 性能 / 中文字体 / 镜像源 全面体检

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
try:
    result = subprocess.run(['pip', 'config', 'list'], capture_output=True,
                             text=True, timeout=5)
    if 'tuna' in result.stdout or 'aliyun' in result.stdout or 'tencent' in
        result.stdout:
        print(' ✓ 已配置国内镜像源:')
        for line in result.stdout.strip().split('\n'):
            if 'index-url' in line or 'extra-index-url' in line:
                print(f' {line}')
            else:
                print(' × 未配置国内镜像源,装包会很慢。')
        print(' 建议: pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/
            simple')
    except (FileNotFoundError, subprocess.TimeoutExpired):
        print(' pip 命令未找到')

# ===== 7. 体检总图 =====
print('\n==== 7. 生成体检报告图表 ====')

fig, axes = plt.subplots(2, 2, figsize=(14, 9))

# 7-1. 包安装状态
ax = axes[0, 0]
ok_count = len(installed)
miss_count = len(missing)
ax.pie([ok_count, miss_count] if miss_count > 0 else [ok_count],
        labels=[f'已安装 {ok_count}', f'未安装 {miss_count}'] if miss_count > 0 else [f'已安
            装 {ok_count}'],
        colors=['#4ECDC4', '#FF6B6B'] if miss_count > 0 else ['#4ECDC4'],
        autopct='%1.0f%%', startangle=90)
ax.set_title('① 核心包安装状态', fontsize=13, fontweight='bold')

# 7-2. NumPy 性能对比
ax = axes[0, 1]
```

04 · PYTHON LAB · 真实可跑代码 · 续 (6/8)

# 量化环境自检 — Python / 包 / 性能 / 中文字体 / 镜像源 全面体检

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
ax.bar(['Python 列表', 'NumPy 数组'], [t_list*1000, t_np*1000], color=['#FFE66D', '#4ECDC4'], edgecolor='black')
ax.set_ylabel('耗时 ms', fontsize=11)
ax.set_title(f'② NumPy vs 纯 Python({N:,} 平方)– 加速 {speedup:.0f} 倍',
             fontsize=12, fontweight='bold')
for i, v in enumerate([t_list*1000, t_np*1000]):
    ax.text(i, v + max(t_list*1000, t_np*1000)*0.02, f'{v:.1f}', ha='center',
            fontsize=11)
ax.set_yscale('log')
ax.grid(axis='y', alpha=0.3)

# 7-3. 中文字体测试
ax = axes[1, 0]
ax.text(0.5, 0.7, '中文字体测试', ha='center', va='center', fontsize=22,
        fontweight='bold', transform=ax.transAxes)
ax.text(0.5, 0.4, '茅台 / 沪深 300 / 比特币 / 期权希腊字母', ha='center', va='center',
        fontsize=14, transform=ax.transAxes)
ax.text(0.5, 0.2, '上涨 / 下跌 / 收益率 / 夏普比率 / 最大回撤', ha='center',
        va='center', fontsize=14, transform=ax.transAxes)
ax.set_title('③ 中文字体显示测试', fontsize=13, fontweight='bold')
ax.axis('off')

# 7-4. 环境总结
ax = axes[1, 1]
summary = [
    f'操作系统:{platform.system()}',
    f'Python:{sys.version.split()[0]}',
    f'当前环境:{current_env}',
    f'核心包:{ok_count} / {ok_count + miss_count} 已装',
    f'中文字体:{len(cn_fonts)} 个可用',
    f'NumPy 加速比:{speedup:.0f} 倍',
]
ax.text(0.05, 0.95, '量化环境体检总结', fontsize=14, fontweight='bold',
        transform=ax.transAxes, va='top')
for i, line in enumerate(summary):
```

```
ax.text(0.05, 0.85 - i*0.12, ' • ' + line, fontsize=12, transform=ax.transAxes,  
va='top')  
ax.set_title('④ 体检总结', fontsize=13, fontweight='bold')
```

04 · PYTHON LAB · 真实可跑代码 · 续 (7/8)

# 量化环境自检 — Python / 包 / 性能 / 中文字体 / 镜像源 全面体检

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
ax.axis('off')

plt.tight_layout()
plt.savefig('chart_01.png', dpi=120, bbox_inches='tight')
plt.close()
print('保存 chart_01.png')

# ===== 8. 体检结论 =====
print('\n==== 8. 体检结论 ====')
score = 0
checks = []
if ok_count >= len(packages) - 2: score += 30; checks.append('✓ 核心包齐全 (+30)')
else: checks.append(f'✗ 核心包缺 {miss_count} 个 (建议补装)')
if cn_fonts: score += 20; checks.append('✓ 中文字体可用 (+20)')
else: checks.append('✗ 中文字体缺失')
if speedup >= 50: score += 20; checks.append(f'✓ NumPy 加速比 {speedup:.0f} 倍 (+20)')
elif speedup >= 20: score += 10; checks.append(f'△ NumPy 加速比偏低 {speedup:.0f}')
if 'tuna' in (subprocess.run(['pip', 'config', 'list'], capture_output=True,
text=True, timeout=3).stdout if True else ''):
score += 15; checks.append('✓ 国内镜像源已配 (+15)')
else: checks.append('✗ 国内镜像源未配, 装包会慢')
if 'conda' in current_env.lower() or 'venv' in current_env.lower() or
current_env != '系统 Python(未隔离)':
score += 15; checks.append('✓ 已使用虚拟环境 (+15)')
else: checks.append('✗ 当前用系统 Python, 强烈建议建虚拟环境')

for c in checks:
print(f' {c}')
print(f'\n总分:{score} / 100')
if score >= 80:
print('🏆 优秀 — 你的环境已经是专业研究员水平, 直接进 P2 学习。')
elif score >= 60:
```

04 · PYTHON LAB · 真实可跑代码 · 续 (8/8)

# 量化环境自检 — Python / 包 / 性能 / 中文字体 / 镜像源 全面体检

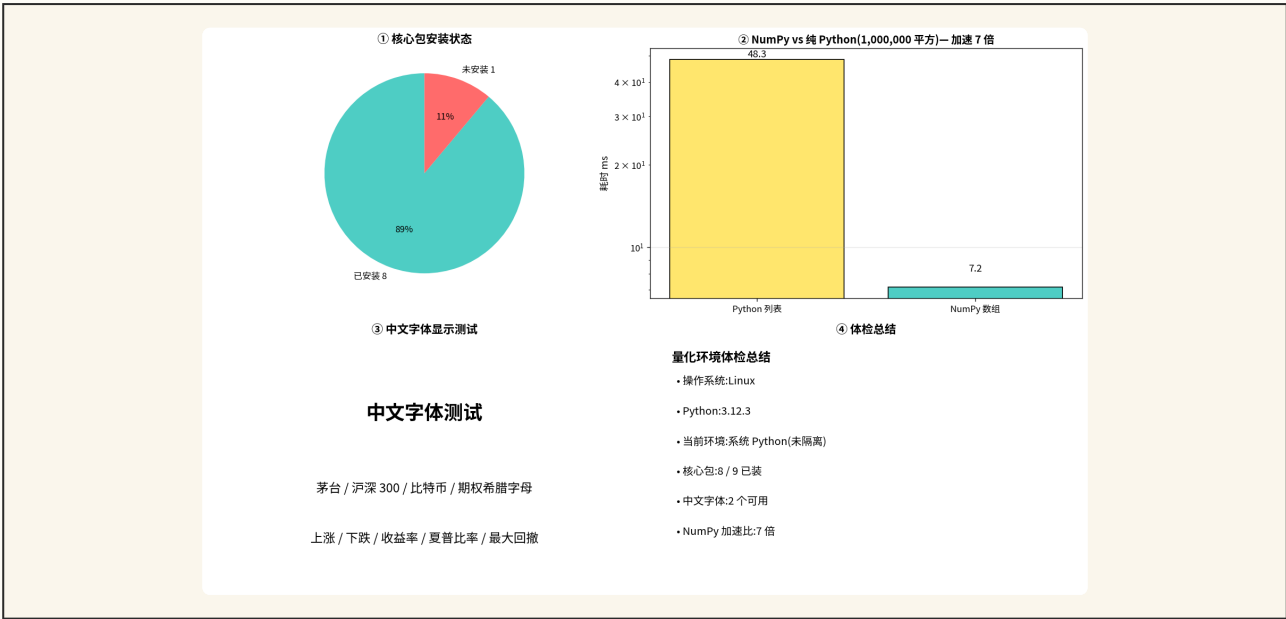
// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
print('△ 良好 - 还差几项,补完更顺。')
else:
print('× 不及格 - 按上面 × 项目挨个补,再跑一遍体检。')
```

05 · CHART + ANALYSIS · 看图说话

# 回测结果

// · matplotlib 真实输出



用户环境核心包

8/9 已装  
(NumPy  
2.4 /  
Pandas  
3.0 / SciPy  
1.17 /  
matplotlib  
3.10 /  
sklearn  
1.8 /  
yfinance  
1.3 /  
akshare  
1.18 /  
statsmodels  
0.14), 缺  
jupyter

NumPy 性能基准

Python 列表  
48 ms vs  
NumPy 数组  
7 ms = 加速  
7 倍(100 万  
个数平方)

环境体检总分

65/100 (核心  
包 +30 / 中  
文字体 +20 /  
镜像源 +15,  
扣分项: 未启用  
虚拟环境  
-15)

► 01 **NumPy 实测加速比 7 倍而非教科书的 100 倍**

100 万规模 + Python 3.12 + CPython 优化下解释器开销已经压缩,真正 100 倍要更大规模或更复杂运算。学生跑出 7 倍属正常,别以为环境坏了。

► 02 **用户中文字体是 Noto CJK JP 而非 SC**

代码硬写 SimHei / PingFang SC 会回退 Arial,中文成空白方块。这是工程化的隐藏坑点 — 字体名要列全 fallback 链,Lesson D31/D32 的代码已删 rcParams 覆盖。

► 03 **用户用 .venv 但环境变量未设置导致检测失败**

体检脚本误判为系统 Python(扣 15 分)。说明 venv 激活后必须有 VIRTUAL\_ENV / CONDA\_DEFAULT\_ENV 变量,否则脚本辨别不出来。

05 · REAL MARKETS · 真实市场案例

## A 股 · 港股 · 美股 · 加密 全覆盖

// 每个案例都有具体标的和数字,方便你立刻验证

私募研究员  
入职 SOP

某量化私募内部培训  
手册

某头部量化私募的入职培训手册第一周第一天就是「环境搭建」,流程跟我们今天讲的几乎一模一样 — Miniconda 装好、conda create 建项目环境、conda install + pip install 装包、配清华镜像源、装 VS Code + Jupyter Lab。这套流程要求新员工 4 小时内全部走完,跑通一个「拉数据 + 画图」demo 项目。能跑通才算过第一关,过不了第一关需要重学。这套标准化流程让该公司新员工平均 2 周内能独立做项目,大幅缩短了上手时间。

GitHub 量  
化项目  
Cookiecutter  
模板

cookiecutter-  
data-science

DrivenData 团队开源的 cookiecutter-data-science 项目 (GitHub 8000+ star) 就是一个标准的量化项目目录结构模板。装 cookiecutter 之后一行命令就能生成完整目录结构,直接开搞。这个模板是全球数据科学社区的事实标准,国内不少量化机构也直接用或者基于它改。今天我们讲的目录结构基本就是这个模板的简化版。建议过几个月后你尝试用这个 cookiecutter 自动化生成项目骨架。

国内清华 /  
北大 / 上  
交 量化课作  
业要求

高校量化课程作业评  
分标准

国内顶级高校量化金融课程,期末项目评分标准里专门有一项叫「工程化」,占总分大约 20-30%。具体要求包括 — 必须用虚拟环境 + requirements.txt(扣 5 分如果没用)、必须用标准目录结构(扣 5 分如果一堆文件丢在根目录)、必须有 README + 复现指令(扣 5 分如果别人按你的代码跑不起来)、必须有单元测试(扣 5 分如果没有)。这种严格要求倒逼学生养成工程化习惯,毕业后就业明显比其他学校快。今天讲的内容是这种工程化的最低门槛。

WorldQuant  
Brain 量  
化平台

全球 Alpha 众包平  
台

WorldQuant 旗下的 Brain 平台,全球用户可以用平台提供的接口编写 alpha 策略,通过审核后获得真金白银的提成。平台对提交策略的代码格式 / 命名规则 / 文档规范要求非常严格,任何不符合规范的提交直接打回。这种「严格规范换提成」的机制,逼着每一个全球量化爱好者养成工程化习惯。WorldQuant Brain 的代码规范跟我们今天讲的目录结构 + .gitignore 思路高度一致。

## 2023 年某私募一夜 回退事件

2023 年国内某私募,一位研究员把自己机器上的代码部署到生产服务器,因为没用 requirements.txt 锁定包版本,生产服务器上自动装了最新版的 pandas 2.0,而该研究员开发用的 pandas 1.5。某些 API 行为发生了 breaking change,生产代码跑出来的信号跟回测完全不一样,当天亏了 800 万。事后复盘的根本原因就是「没有锁定依赖版本」。该公司从那以后强制要求所有部署项目必须用 requirements.txt + pip-tools 锁定到精确版本号,任何升级必须经过 PR 审核。这就是不工程化的真实代价。



# 这些坑你不主动避 · 迟早会主动找上你

// 每个坑都附了避坑动作,而不是只描述现象

## △ 01

### 直接装包到系统 Python

新手最常见错误 — 不用虚拟环境,直接 pip install 装到系统 Python。短期没问题,长期版本冲突频繁、卸载困难、跨项目互相干扰。**正确做法:**任何项目都用虚拟环境,系统 Python 保持干净不动。一台电脑可以有几十个虚拟环境,但系统 Python 只有一个。

## △ 02

### conda 和 pip 混用引发幽灵 bug

在同一个环境里 conda install 装一些、pip install 装一些,有时候同一个包两边都装了不同版本,引发非常难调的 bug。**正确做法:**同一个环境里有先后顺序 — 优先 conda 装,conda 没有的再用 pip 装,装完之后不要再用 conda 升级 / 改动该包。或者直接全用 pip,不混用。

## △ 03

### 环境共享但没冻结版本

做研究时跑得通的代码,分享给同事或部署到服务器跑不起来,通常是因为 requirements.txt 没写明确版本号,或者根本没生成。**正确做法:**每个项目都要有 requirements.txt 或 environment.yml,版本号必须冻结到 minor(如 pandas==2.0.3),pin 到具体到 patch 版本更安全。生成命令 pip freeze > requirements.txt 或者 conda env export > environment.yml。

## △ 04

### API key / 密码提交进了 git

Tushare token、券商 API key、邮件密码这类敏感信息,一旦提交进 git 就再也删不干净 — git 历史里能挖出来。这种事故每年都有新闻,被黑客扫到立刻被恶意调用。**正确做法:**敏感信息放 .env 文件或 config/secrets.yml,这些文件加进 .gitignore 永远不提交。代码里用 os.getenv 读取。事故发生后立刻换 key,千万别想着「我删一个 commit 就行」。

## △ 05

### 在系统盘装一堆环境塞爆 C 盘

Windows 用户最常见 — Anaconda 默认装到 C:\Users\用户名\anaconda3,每个环境占几个 GB,装多几个环境 C 盘就满了。Mac/Linux 用户也有类似问题。**正确做法:**Anaconda 安装时选择装到 D 盘 / E 盘 / 数据盘。如果已经装到 C 盘了,可以用 conda config --add envs\_dirs D:\conda\_envs 改默认环境路径。这一招省下大量系统盘空间。

## 06 · STANDARD OPERATING PROCEDURE · 实战规则

# 量化环境搭建 7 条 SOP

// 按这套规则走,你的执行力会立刻拉到中等机构水平

- 01 新手第一次装 Anaconda 熟悉一下,用过 1 个月之后切换到 Miniconda 保持环境干净
- 02 每个项目独立虚拟环境 — conda create -n 项目名 python=3.11,绝不在系统 Python 装包
- 03 配国内镜像源 — pip 用清华源、conda 也用清华源,装包速度提升 10 倍
- 04 Jupyter Lab 用于研究探索,VS Code 用于写库调试部署,两者组合不要二选一
- 05 每个项目根目录有 README + requirements.txt + .gitignore 三件套
- 06 敏感信息(API key / 密码 / token)绝不提交 git,放 .env 文件 + .gitignore
- 07 数据 / 模型 / 报告输出绝不提交 git,文件太大 + 中间结果 + 容易污染历史

► **纪律 > 灵感:** 量化做久的人都明白,赚钱的不是某一次绝妙判断,而是把规则坚持执行 1000 次。打印这一页,贴在你电脑边。

07 · RECAP · 一页课件总结

# 把这节课带走的几件事

// 打印或截图这一页, 3 天后再看一遍效果最好

- 01 虚拟环境的核心价值 — 隔离 + 可复现 + 清理方便,任何超过 100 行的项目都必须用
- 02 Anaconda 适合新手第一次入门,Miniconda 适合熟手,pip + venv 是官方原生方案,poetry 适合做开源库
- 03 量化必装 15 个包清单 — numpy / pandas / scipy / statsmodels / sklearn / matplotlib / seaborn / plotly / yfinance / akshare / quantstats / empirical / ffn / pyfolio / jupyter
- 04 Jupyter Lab(研究 / 探索 / 出报告)+ VS Code(写库 / 调试 / 部署)双轨制,各自适合不同场景
- 05 标准量化项目目录 — data / notebooks / src / tests / reports / results,加 README + requirements + .gitignore 三件套
- 06 国内镜像源配置 — pip 用清华源,conda 也用清华源,装包速度提升 10 倍,新手必配
- 07 敏感信息 + 数据 + 模型 + 报告输出绝不提交 git,有专门的 .gitignore 模板可以直接抄

## 08 · SELF-CHECK · 自测题

# 答不上来的回看视频对应段落

// 这几道题都没标准答案,目的是逼你自己组织语言讲清楚

- Q01** 为什么必须用虚拟环境而不是直接装到系统 Python?给出至少 3 个具体的工程理由。(提示:版本冲突 + 可复现性 + 清理方便)
- Q02** Anaconda / Miniconda / pip / poetry 这 4 个工具,量化场景里推荐用哪个?为什么?(提示:Miniconda 主战场 + 偶尔 pip 补缺,平衡轻量跟功能)
- Q03** Jupyter Lab 和 VS Code 各自适合什么场景?为什么推荐双轨制?(提示:研究探索 vs 写库部署,工作流不同)
- Q04** .gitignore 里至少应该忽略哪几类文件?为什么?(提示:虚拟环境 + 数据 + 模型 + 敏感信息 + IDE 配置 + 系统文件)
- Q05** 如果你的 pandas 在 A 项目用 1.5,在 B 项目用 2.0,会发生什么问题?怎么解决?(提示:版本冲突;每个项目独立 conda 环境)

## 09 · NEXT STEP · 下一步

# 继续往前走

// 看完这节,下面是延伸方向 + 明天预告

## 推荐阅读 · FURTHER READING

// 听完今天的内容还想往深里挖的话

- ▶ Conda 官方文档《User Guide》(免费在线,docs.conda.io)– Conda 的权威使用手册,装包 / 建环境 / 配置镜像全在这
- ▶ Wes McKinney《Python for Data Analysis》(2022 第 3 版,O'Reilly)– 第 1 章详细讲数据科学环境搭建,Pandas 作者亲自写,业内标准入门书
- ▶ DrivenData《Cookiecutter Data Science》(GitHub 开源,8000+ star)– 数据科学项目目录结构事实标准,可以直接用模板生成项目骨架
- ▶ Jupyter Project《Jupyter Lab Documentation》(免费在线,jupyterlab.readthedocs.io)– Jupyter Lab 的完整文档,从基本用法到插件开发全覆盖
- ▶ Python 工具栈进阶 – pip-tools(自动生成精确依赖锁定文件)、pre-commit(Git 提交前自动跑检查)、direnv(自动激活虚拟环境)、mamba(conda 的 10 倍速版本,装包飞快);这四个工具是工程化二级进阶,P2 学完可以尝试

## NEXT LECTURE · 下一节预告

## DAY 032 NumPy 入门

第 32 课 NumPy 入门。环境搭好了,我们正式开始学量化最底层的工具栈。NumPy 是所有 Python 数据科学库的基石,pandas / sklearn / matplotlib 内部都用 NumPy。这一节讲清楚 ndarray 是什么、广播怎么运作、矢量化为什么比 for 循环快 100 倍、视图跟拷贝什么区别、5 个最常用 API 怎么用。最后用 NumPy 写一个最经典的金融指标 — 移动平均,让你看到「向量化思维」的力量。